

Jenkins pipeline

Tasks:

1 Create one Declarative pipeline job

1. Source Code: <https://github.com/betavins/spring3-mvc-maven-xml-hello-world-1.git>

Task 1: Create One Declarative Pipeline Job

Objective :

To create a Jenkins Declarative Pipeline job that builds a Maven-based Spring MVC project.

Source Code :

Repository:

`https://github.com/betavins/spring3-mvc-maven-xml-hello-world-1.git`

Prerequisites

- Jenkins installed
- Maven installed and configured in Jenkins (Global Tool Configuration)
- Git installed
- Java configured

The pipeline should:

- Pull source code from GitHub
- Build using Maven
- Perform static code analysis using SonarQube
- Validate Quality Gate
- Upload artifact to Nexus repository

Tools Used

Tool	Purpose
GitHub	Source Code Management
Jenkins	CI Pipeline Automation
Maven	Build Tool
SonarQube	Static Code Analysis
Nexus Repository	Artifact Storage

Maven Tool Configuration

Manage Jenkins → Global Tool Configuration

- Name: `maven`
- Version: `3.9.12`
- Install Automatically: Enabled

SonarQube Configuration

Manage Jenkins → Configure System → SonarQube Servers

- Name: `sonarqube`
- Server URL: `http://localhost:9000`

Maven installations

Maven installations ^ Edited

+ Add Maven

Maven

Name

maven

☒ Install automatically ?

Install from Apache

Version

3.9.12

+ Add Installer

+ Add Maven

SonarQube Scanner installations

SonarQube Scanner installations ^ Edited

+ Add SonarQube Scanner

SonarQube Scanner

Name

sonar-scanner

SONAR_RUNNER_HOME

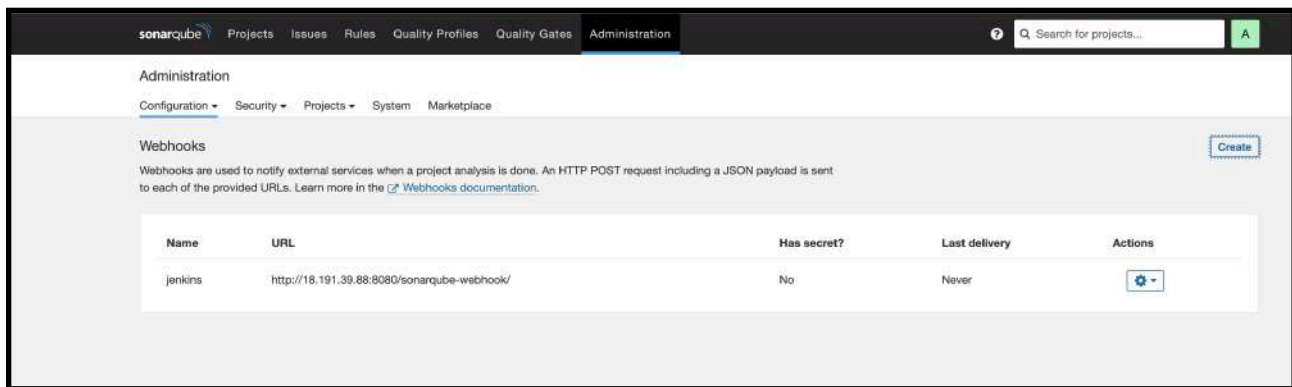
/opt/sonar-scanner

☐ Install automatically ?

+ Add SonarQube Scanner

Webhook configured in SonarQube:

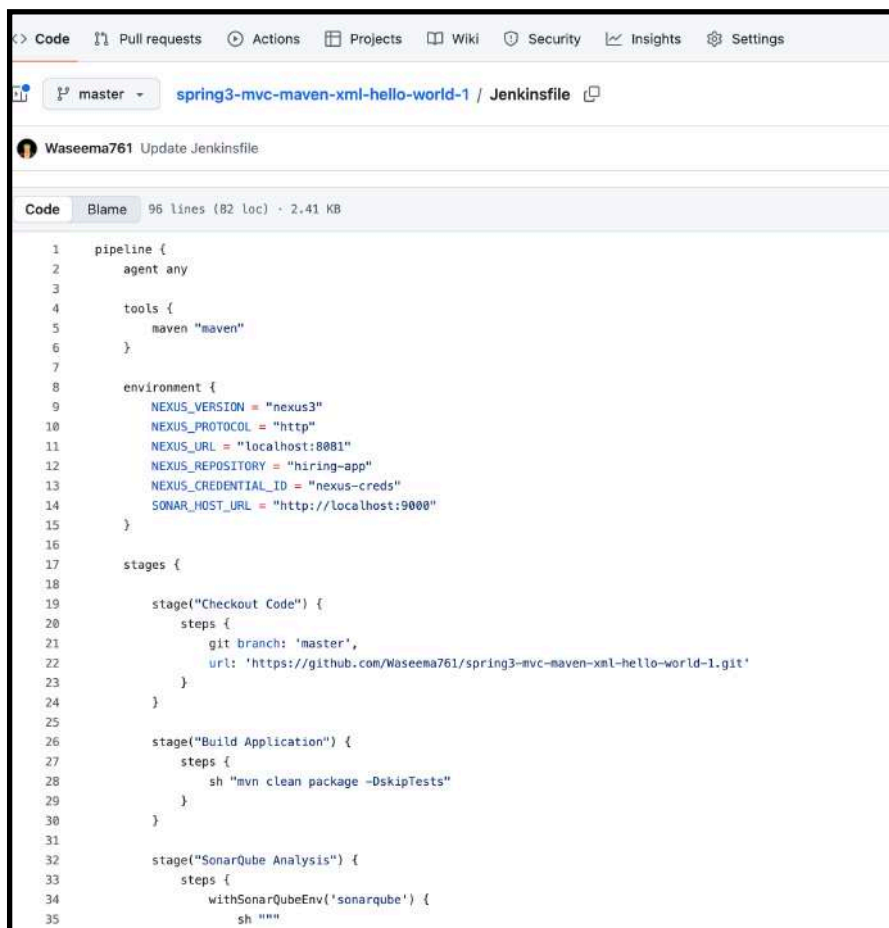
<http://18.191.39.88:8080/sonarqube-webhook/>



Nexus Configuration

- Nexus URL: `http://18.191.39.88:8081`
- Repository Name: `hiring-app`
- Jenkins Credential ID: `nexus-creds`

Jenkinsfile (Declarative Pipeline)



Pipeline Workflow

GitHub



Jenkins (Checkout)



Maven Build



SonarQube Analysis



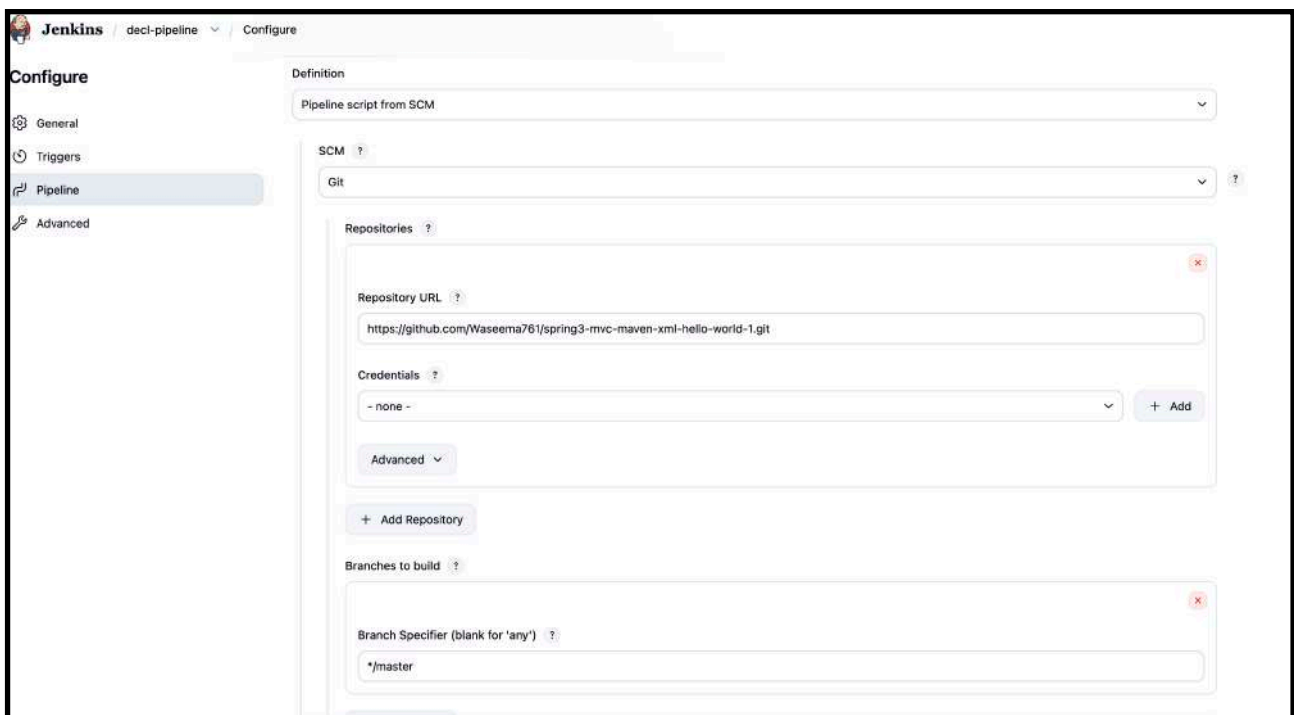
Quality Gate Validation



Nexus Artifact Upload

—> now inside Jenkins create a job pipeline

—-> add the scm of git



The screenshot shows the Jenkins 'Configure' page for a pipeline named 'decl-pipeline'. The 'Definition' section is set to 'Pipeline script from SCM'. Under the 'SCM' section, 'Git' is selected. A single repository is configured with the URL 'https://github.com/Waseema761/spring3-mvc-maven-xml-hello-world-1.git' and no credentials. The 'Branches to build' section is set to '*/master'. The left sidebar shows the 'Configure' menu with options for General, Triggers, Pipeline, and Advanced.

Jenkins decl-pipeline Configure

Configure

- General
- Triggers
- Pipeline
- Advanced

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Waseema761/spring3-mvc-maven-xml-hello-world-1.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

—> build the job



Output

- WAR file generated:

target/ncodeit-hello-world-3.0.war

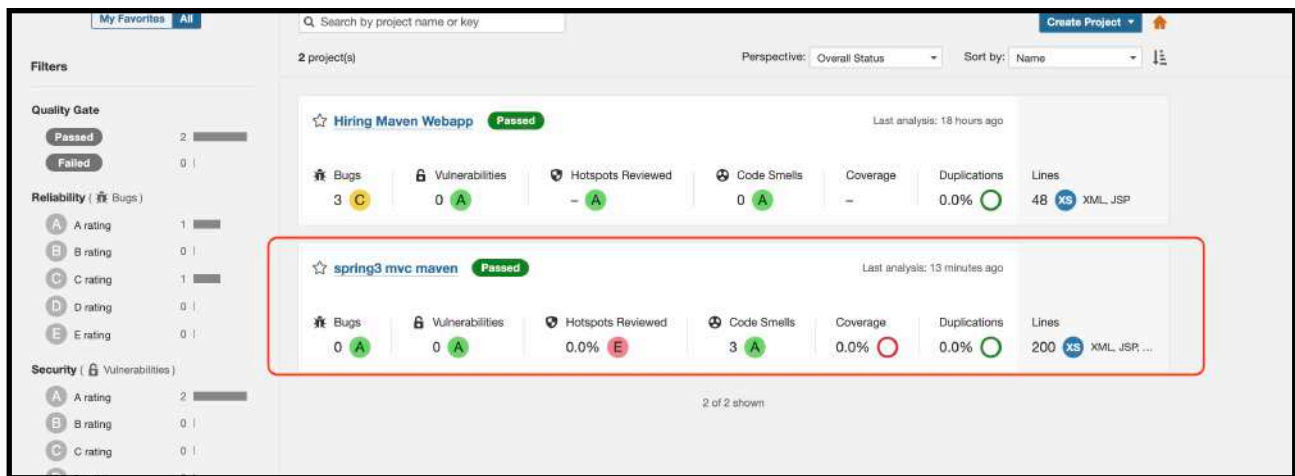
Artifact stored in Nexus:

```
hiring-app
├── com
│   └── ncodeit
│       ├── ncodeit-hello-world
│       └── <BUILD_NUMBER>
```

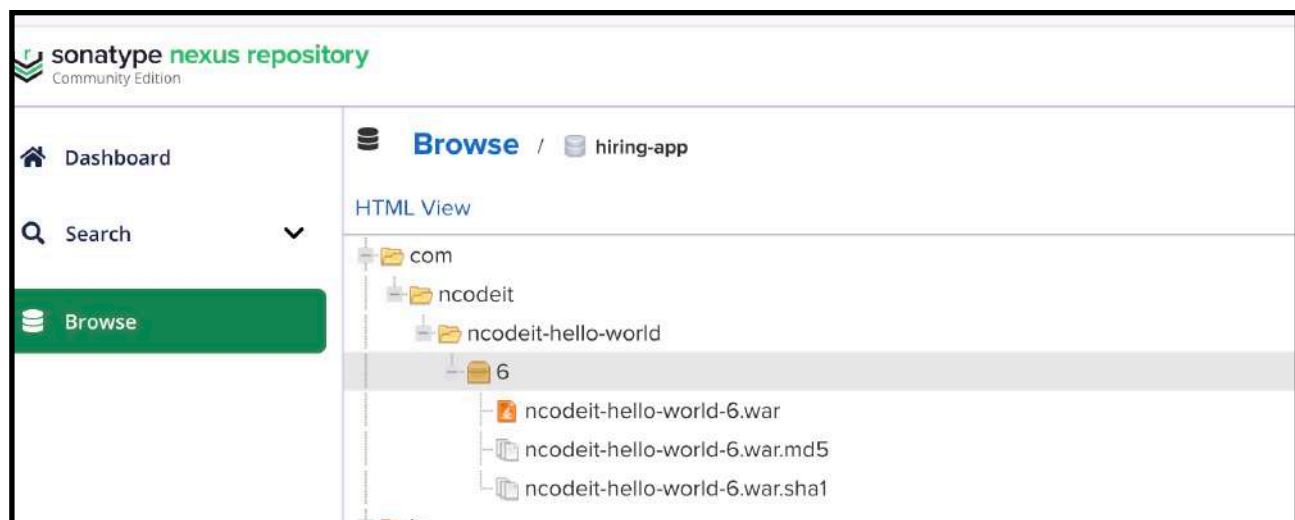
•

Verify :

Inside sonarqube projects :



Inside the nexus artifact hiring-app repo



Conclusion

The Declarative Pipeline successfully implements Continuous Integration (CI) by:

- Automating code build
- Performing static code analysis
- Enforcing quality standards
- Storing versioned artifacts in Nexus repository

Task 2: Create Scripted Pipeline Job

Objective

To implement the same Continuous Integration (CI) process using a **Scripted Jenkins Pipeline** instead of Declarative syntax.

The pipeline performs:

- Source code checkout from GitHub
- Build using Maven
- Static code analysis using SonarQube
- Quality Gate validation
- Artifact publishing to Nexus repository

Tools Used

Tool	Purpose
GitHub	Source Code Management
Jenkins	CI Automation
Maven	Build Tool
SonarQube	Code Quality Analysis
Nexus	Artifact Repository

Source Code Repository

`https://github.com/Waseema761/spring3-mvc-maven-xml-hello-world-1.git`

Step 1: Create New Jenkins Job

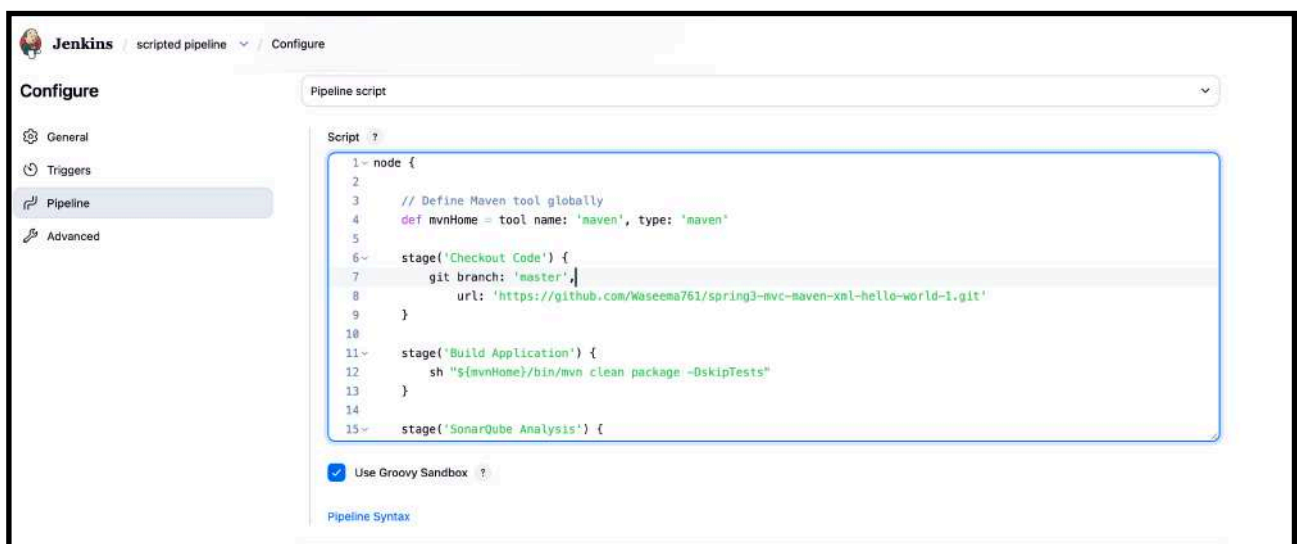
1. Go to Jenkins Dashboard

2. Click **New Item**
3. Enter job name:
4. Select **Pipeline**
5. Click **OK**

Step 2: Configure Pipeline

Scroll to **Pipeline Section**

Set:



Pipeline Execution Flow

```
GitHub
  ↓
Jenkins (node block)
  ↓
Maven Build
  ↓
SonarQube Analysis
```

↓
Quality Gate Validation
↓
Nexus Artifact Upload

Key Differences from Declarative Pipeline

Declarative	Scripted
Uses <code>pipeline {}</code>	Uses <code>node {}</code>
Structured syntax	Pure Groovy syntax
Easier for beginners	More flexible
Tools declared in <code>tools {}</code>	Tools defined using <code>tool</code> step

—-> build the job

The screenshot shows the Jenkins web interface for a 'scripted pipeline' job. The left sidebar contains navigation links: Status, Changes, Build Now (highlighted), Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main content area displays the 'Stage View' for the 'scripted pipeline' job, which is currently in a failed state. A table shows the stages: Checkout Code, Build Application, SonarQube Analysis, Quality Gate Check, and Publish Artifact to Nexus. The 'SonarQube Analysis' stage is highlighted in red, indicating a failure. Below the table, there are 'Permalinks' for the last build, last failed build, last unsuccessful build, and last completed build. The bottom left shows a list of builds for today, with build #2 at 12:46 PM and build #1 at 12:45 PM.

Stage	Average stage times: (full run time: ~18s)
Checkout Code	342ms
Build Application	4s
SonarQube Analysis	5s
Quality Gate Check	101ms
Publish Artifact to Nexus	208ms

Build	Checkout Code	Build Application	SonarQube Analysis	Quality Gate Check	Publish Artifact to Nexus
#2 Mar 03 18:16	230ms	4s	11s	101ms (paused for 1s)	208ms
#1 Mar 03 18:15	455ms	4s	95ms failed		

Permalinks

- Last build (#1), 1 min 5 sec ago
- Last failed build (#1), 1 min 5 sec ago
- Last unsuccessful build (#1), 1 min 5 sec ago
- Last completed build (#1), 1 min 5 sec ago

Builds

Today

- #2 12:46 PM
- #1 12:45 PM

Output

Generated Artifact:

target/ncodeit-hello-world-3.0.war

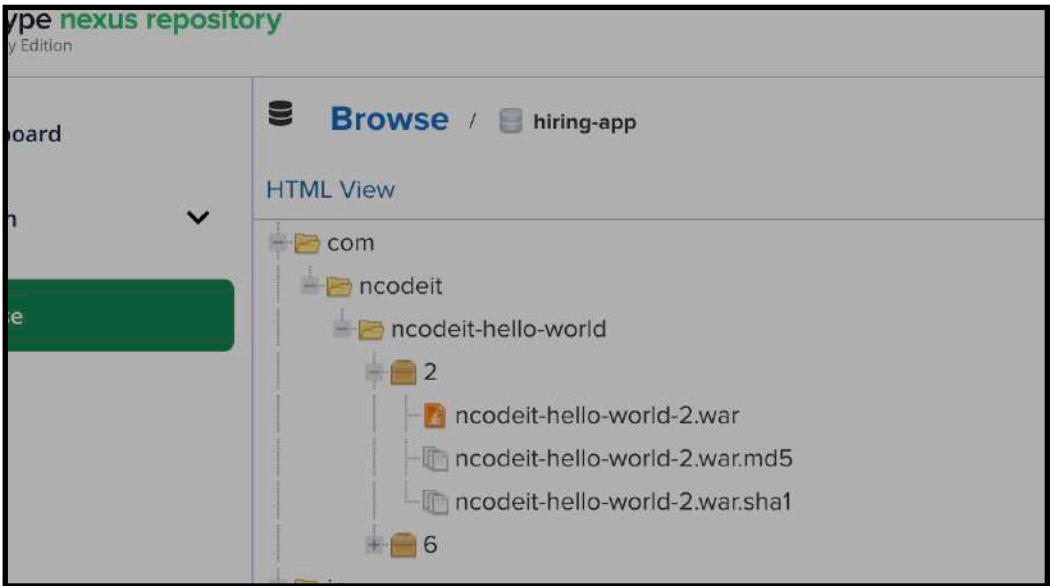
Stored in Nexus:

```
hiring-app
├── com
│   └── ncodeit
│       ├── ncodeit-hello-world
│           └── <BUILD_NUMBER>
```

Verify:



Nexus:



Conclusion

The Scripted Pipeline successfully implements Continuous Integration using Groovy-based Jenkins scripting. It provides greater flexibility compared to Declarative Pipeline while achieving the same CI functionality.

Task 3: Create One Parameterized Pipeline Job

This is important for interviews also 💪

Objective

To create a Jenkins pipeline job that accepts input parameters at build time.

Instead of hardcoding:

- Branch
- Sonar project key
- Environment

We will pass them as parameters.

What We Will Parameterize

We will add:

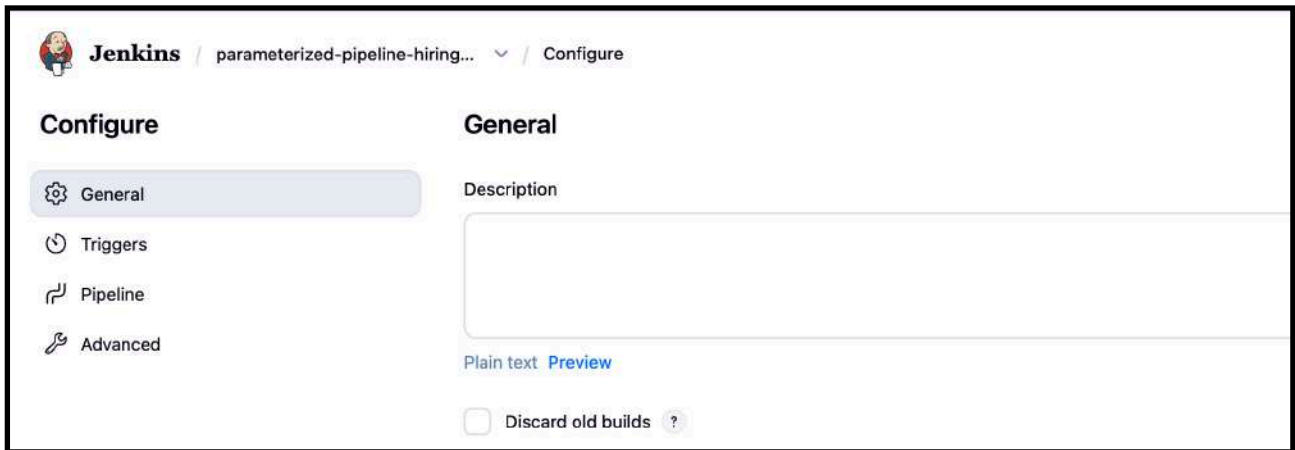
Parameter	Purpose
BRANCH_NAME	Git branch to build
SONAR_PROJECT_KEY	Sonar project key
VERSION	Version for Nexus upload

Step 1: Create New Job

1. Jenkins Dashboard
2. Click **New Item**
3. Name:

parameterized-pipeline-hiring-app

4. Select **Pipeline**
5. Click OK



The screenshot shows the Jenkins 'Configure' page for a job named 'parameterized-pipeline-hiring-app'. The breadcrumb navigation at the top reads 'Jenkins / parameterized-pipeline-hiring... / Configure'. On the left, under the 'Configure' heading, there is a sidebar with four options: 'General' (selected), 'Triggers', 'Pipeline', and 'Advanced'. The main area is titled 'General' and contains a 'Description' text area, which is currently empty. Below the text area, there is a 'Plain text' label and a 'Preview' link. At the bottom of the main area, there is a checkbox labeled 'Discard old builds' with a help icon (?) next to it.

Step 2: Enable Parameters

Inside Job Configuration:

✓ Check **This project is parameterized**

Add:

String Parameter 1

Name:

BRANCH_NAME

Default:

master

☐ Pipeline speed/durability override ?

☐ Preserve stashes from completed builds ?

☒ This project is parameterized ?

String Parameter ?

Name ?

BRANCH_NAME

Default Value ?

master

Description ?

Plain text [Preview](#)

☐ Trim the string ?

String Parameter 2

Name:

SONAR_PROJECT_KEY

Default:

hiring-app

String Parameter ?

Name ?

SONAR_PROJECT_KEY

Default Value ?

hiring-app

Description ?

String Parameter 3

Name:

VERSION

Default:

1.0



The screenshot shows the 'String Parameter' configuration page in Jenkins. The 'Name' field is set to 'VERSION'. The 'Default Value' field is set to '1.0'. The 'Description' field is empty. There are help icons (?) next to the 'Name', 'Default Value', and 'Description' labels.

Parameterized Declarative Pipeline Code

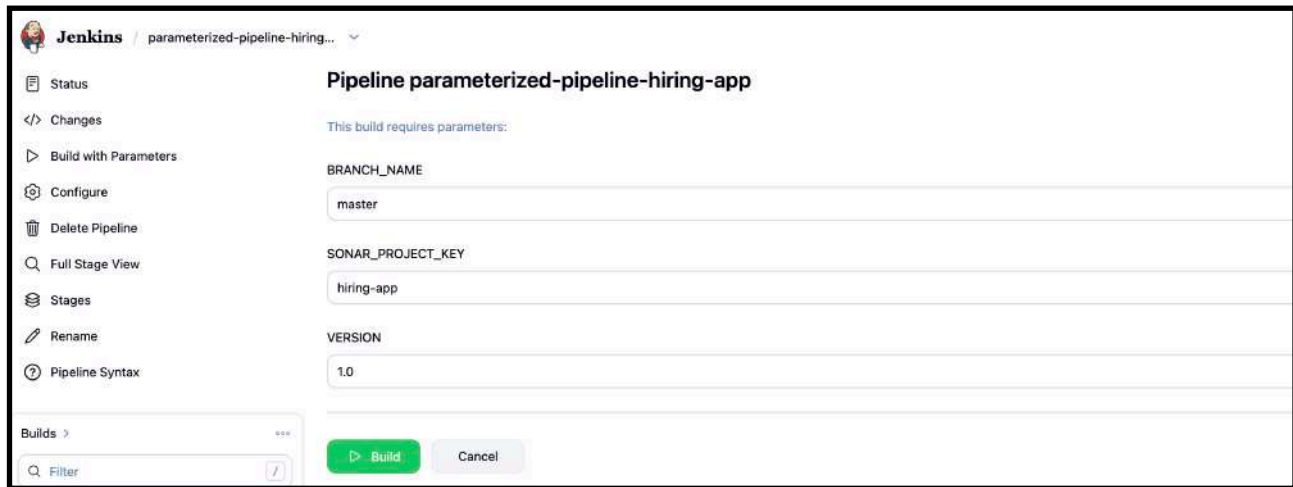


The screenshot shows the Jenkins Pipeline configuration page. The 'Definition' dropdown is set to 'Pipeline script'. The 'Script' field contains the following Groovy code:

```
1 pipeline {
2   agent any
3
4   tools {
5     maven "maven"
6   }
7
8   parameters {
9     string(name: 'BRANCH_NAME', defaultValue: 'master', description: 'Git Branch to Build')
10    string(name: 'SONAR_PROJECT_KEY', defaultValue: 'hiring-app', description: 'Sonar Project Key')
11    string(name: 'VERSION', defaultValue: '1.0', description: 'Artifact Version')
12  }
13
14  environment {
15    NEXUS_URL = "localhost:8081"
16  }
17 }
```

Below the script field, there is a checkbox labeled 'Use Groovy Sandbox' which is checked. At the bottom left, there is a link for 'Pipeline Syntax'.

—-> build with parameters



The image shows the Jenkins configuration page for a pipeline named "parameterized-pipeline-hiring-app". The left sidebar contains navigation links: Status, Changes, Build with Parameters (selected), Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main area displays the parameters for the build:

- BRANCH_NAME: master
- SONAR_PROJECT_KEY: hiring-app
- VERSION: 1.0

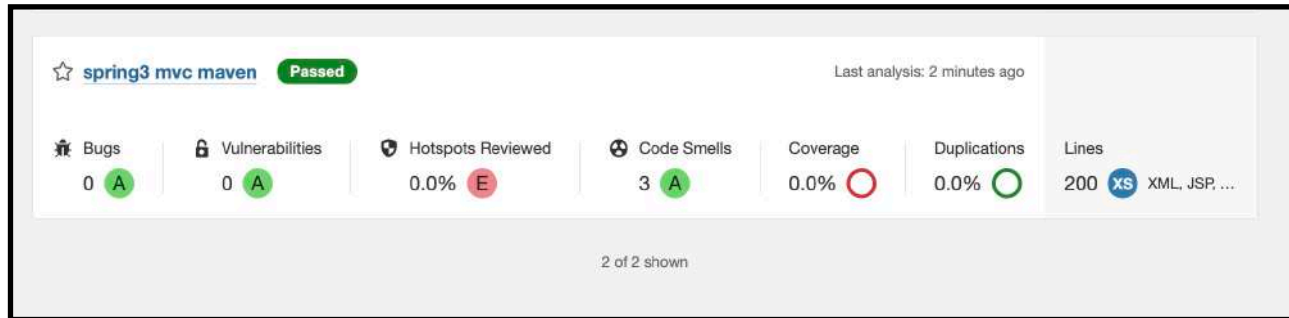
At the bottom, there is a "Build" button and a "Cancel" button. A "Builds" section at the bottom left shows a filter and a list of builds.

—> build

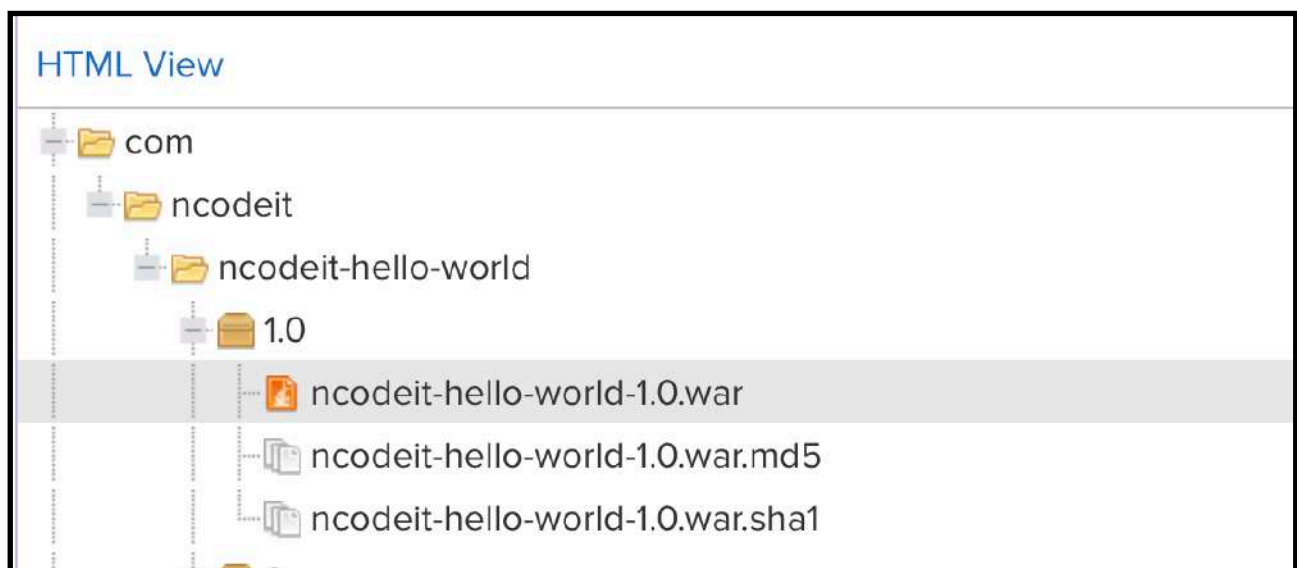


Verify:

Sonar



Nexus:



Pipeline Flow

```
graph TD; A[User inputs parameters] --> B[Checkout selected branch]; B --> C[Build application]; C --> D[Run Sonar analysis]; D --> E[Check quality gate];
```

↓

Upload versioned artifact to Nexus

Result

Different builds can produce different versions:

Example:

- Version 1.0
- Version 2.0
- Version 3.1

All stored separately in Nexus repository.

Conclusion

The Parameterized Pipeline allows dynamic input during build time, making the pipeline reusable and flexible for different branches, environments, and versions.

Task 4: Write Sample Skeleton for Declarative and Scripted Pipelines

Objective

To understand and demonstrate the basic structural difference between:

- Declarative Pipeline
- Scripted Pipeline

Declarative Pipeline Skeleton

Declarative pipeline uses:

```
pipeline { }
```

It has predefined structure.

Sample Declarative Pipeline Skeleton

```
pipeline {  
  
    agent any  
  
    tools {  
        // Optional tools  
        maven "maven"  
    }  
  
    environment {  
        // Optional environment variables  
        APP_NAME = "SampleApp"  
    }  
  
    parameters {  
        // Optional parameters  
        string(name: 'VERSION', defaultValue: '1.0')  
    }  
  
    stages {  
  
        stage("Stage 1") {  
            steps {  
                echo "This is Stage 1"  
            }  
        }  
  
        stage("Stage 2") {  
            steps {  
                echo "This is Stage 2"  
            }  
        }  
    }  
}
```

```

    }
  }

  post {
    success {
      echo "Pipeline executed successfully"
    }

    failure {
      echo "Pipeline failed"
    }
  }
}

```

Structure of Declarative Pipeline

```

pipeline
├── agent
├── tools (optional)
├── environment (optional)
├── parameters (optional)
├── stages
│   ├── stage
│   └── stage
└── post (optional)

```

Scripted Pipeline Skeleton

Scripted pipeline uses:

```
node { }
```

It is based on pure Groovy syntax.

Sample Scripted Pipeline Skeleton

```
node {  
  
    stage('Stage 1') {  
        echo "This is Stage 1"  
    }  
  
    stage('Stage 2') {  
        echo "This is Stage 2"  
    }  
  
    try {  
  
        stage('Stage 3') {  
            echo "This is Stage 3"  
        }  
  
    } catch (Exception e) {  
        echo "Error occurred"  
        throw e  
    }  
  
    echo "Pipeline Completed"  
}
```

Structure of Scripted Pipeline

```
node  
├── stage  
├── stage  
├── try-catch (optional)  
└── custom Groovy logic
```

Key Differences

Declarative	Scripted
Uses <code>pipeline {}</code>	Uses <code>node {}</code>
Fixed structure	Flexible structure
Easier to read	More powerful
Less Groovy code	Pure Groovy scripting

Declarative pipeline is recommended for most CI/CD use cases because it is structured and easier to maintain. Scripted pipeline is used when advanced customization and complex logic are required.

Task 5 – Create one Declarative Jenkins Pipeline Job

Source Code

Repository used:

https://github.com/betawins/sabear_simplecutomerapp.git

Objective

Create a **Declarative Jenkins Pipeline** that:

- Clones source code from GitHub
- Builds the application using Maven
- Performs code analysis using SonarQube
- Checks the Quality Gate
- Uploads the artifact to Nexus Repository

Prerequisites

The following configurations were already completed in previous tasks:

1. Jenkins installed and running.
2. Maven configured in Jenkins.
3. SonarQube server configured in Jenkins.
4. Nexus Repository installed and running.
5. Nexus repository created:

`hiring-app-snapshot`

6. Nexus credentials added in Jenkins:

Credential ID: `nexus-creds`

7. GitHub repository available:

`https://github.com/betawins/sabear_simplecutomerapp.git`

Step 1 – Create Jenkins Pipeline Job

1. Login to Jenkins.
2. Click:

New Item

—> select pipeline and create job

Step 2 – Configure Pipeline

Definition

Pipeline script from SCM

SCM ?

Git

Repositories ?

Repository URL ?

https://github.com/Waseema761/sabear_simplecutomerapp.git

Credentials ?

- none -

+ Add

Advanced

+ Add Repository

Step 3 – Run the Pipeline

Click:

Build Now



Jenkins executes the following stages:

1. Checkout Code
2. Build Application
3. SonarQube Analysis
4. Quality Gate Check
5. Publish Artifact to Nexus

Final Verification

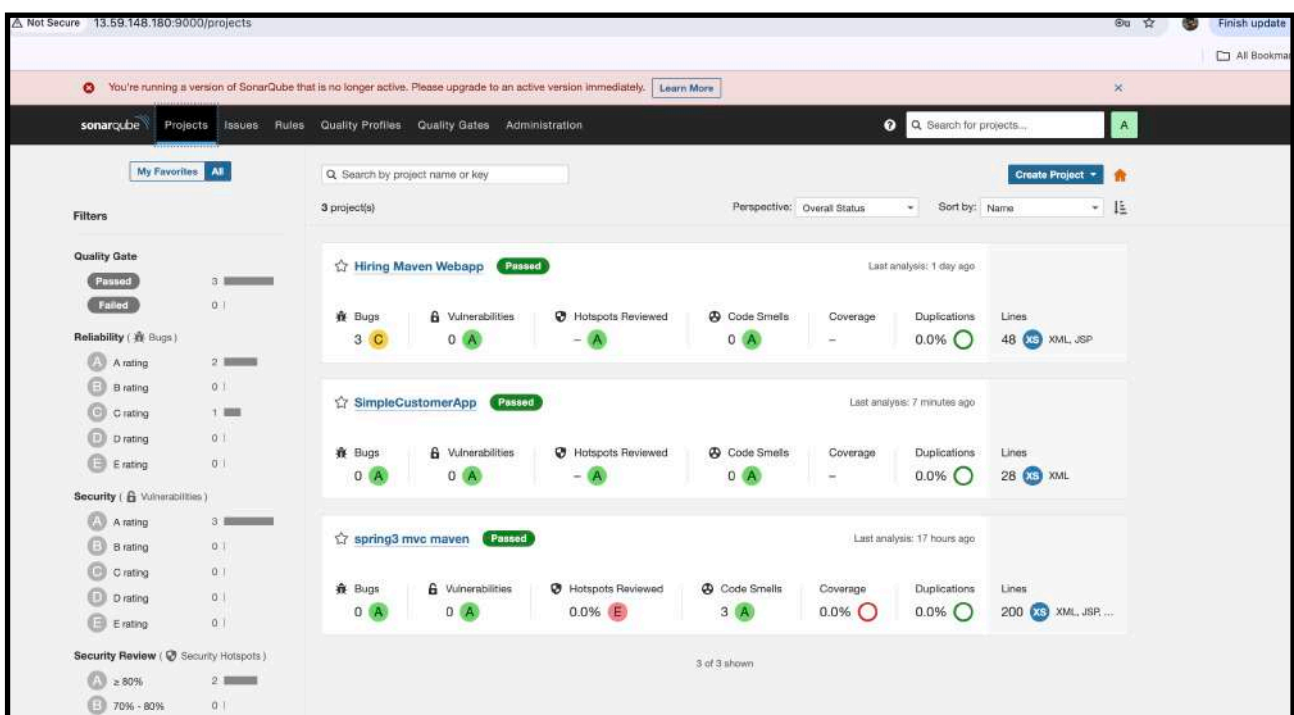
SonarQube Verification

Open SonarQube dashboard:

`http://13.59.148.180:9000`

Verify:

Project created successfully
Quality Gate Status: PASSED
Code analysis report generated



Nexus Artifact Verification

Open Nexus repository:

`http://13.59.148.180:8081`

Navigate to:

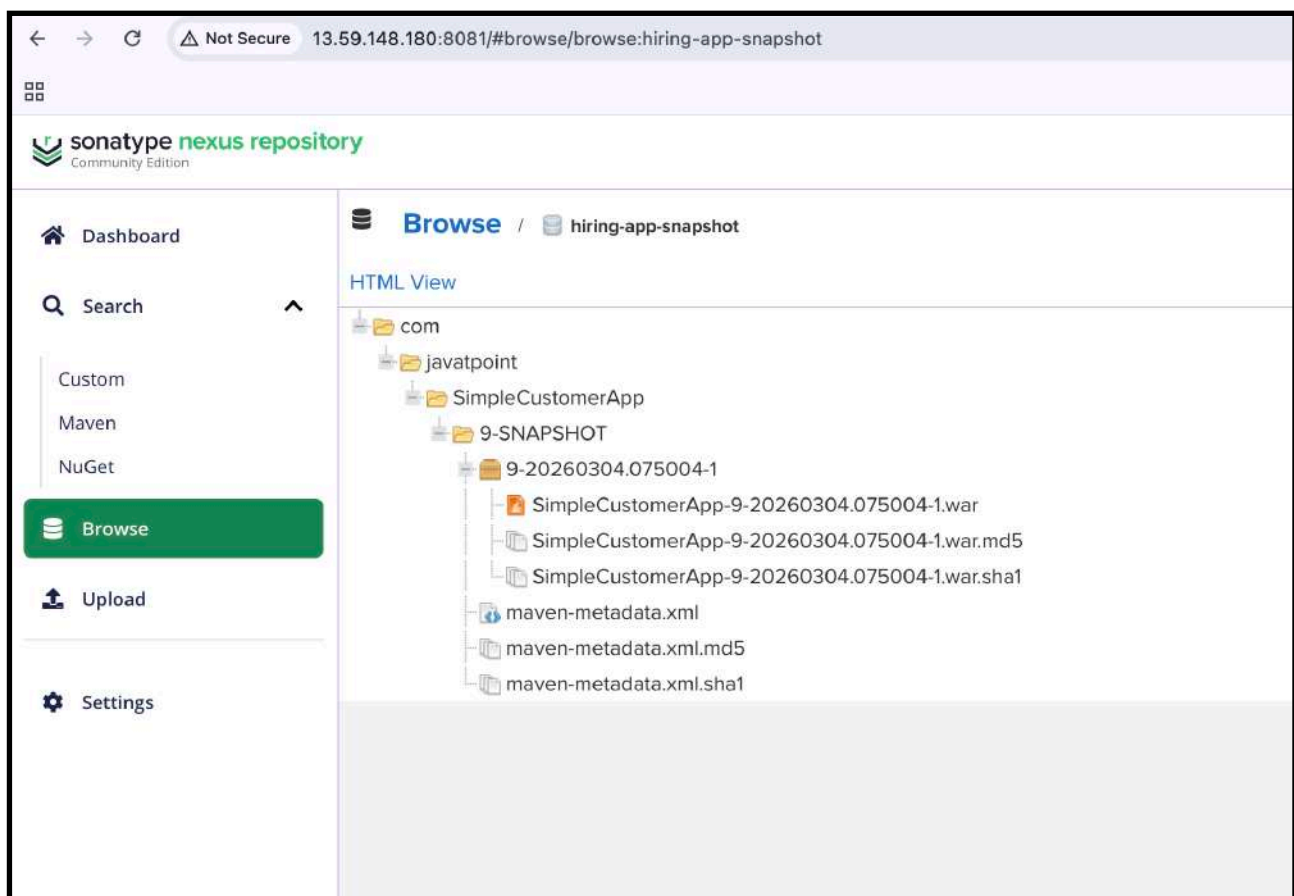
Browse

- hiring-app-snapshot
- com
- javatpoint
- SimpleCustomerApp
- SNAPSHOT version

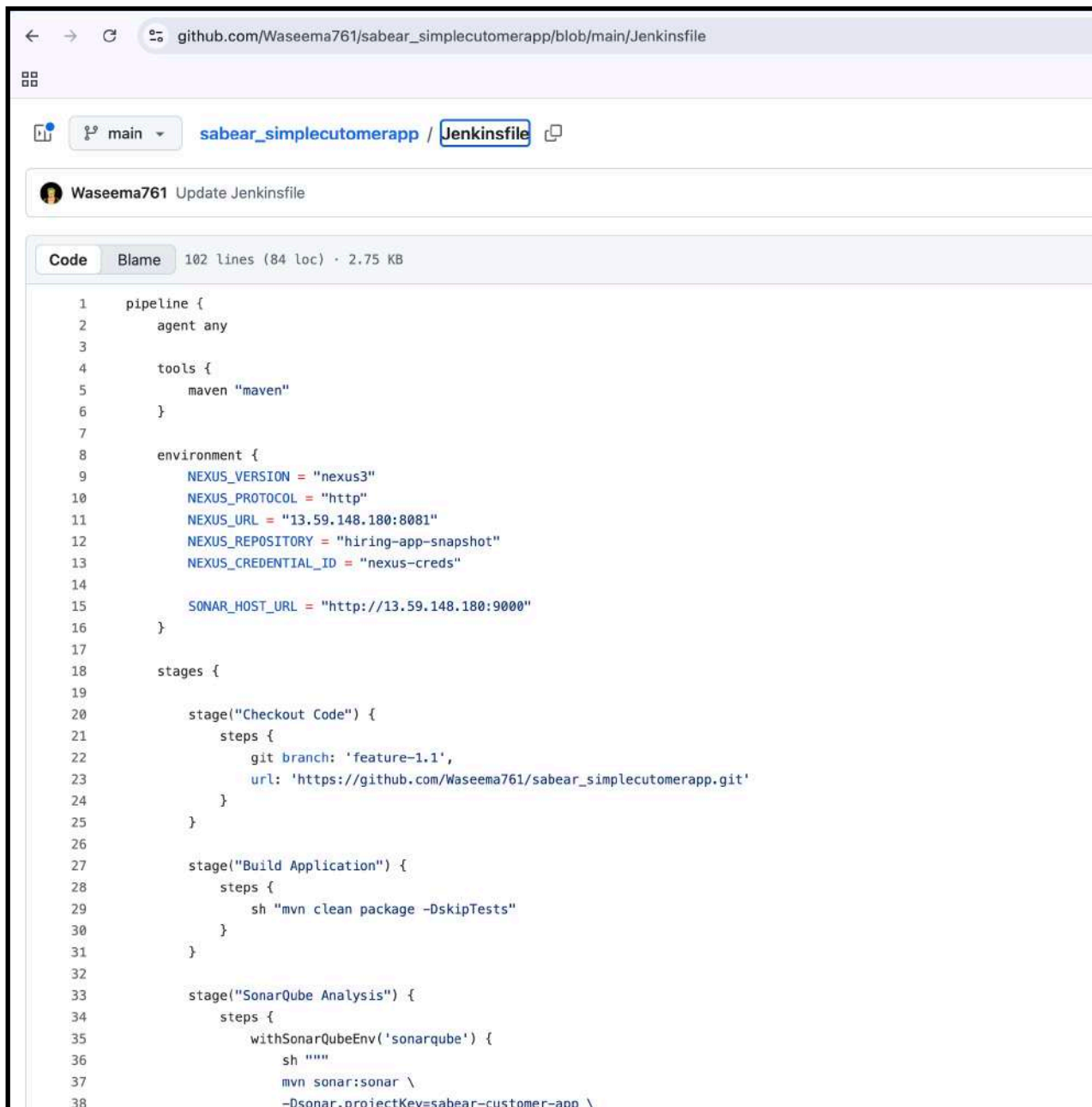
Verify artifact:

`SimpleCustomerApp-*-SNAPSHOT.war`

This confirms the artifact was uploaded successfully from Jenkins.



Pipeline used from source code :



```
1 pipeline {
2   agent any
3
4   tools {
5     maven "maven"
6   }
7
8   environment {
9     NEXUS_VERSION = "nexus3"
10    NEXUS_PROTOCOL = "http"
11    NEXUS_URL = "13.59.148.180:8081"
12    NEXUS_REPOSITORY = "hiring-app-snapshot"
13    NEXUS_CREDENTIAL_ID = "nexus-creds"
14
15    SONAR_HOST_URL = "http://13.59.148.180:9000"
16  }
17
18  stages {
19
20    stage("Checkout Code") {
21      steps {
22        git branch: 'feature-1.1',
23          url: 'https://github.com/Waseema761/sabear_simplecutomerapp.git'
24      }
25    }
26
27    stage("Build Application") {
28      steps {
29        sh "mvn clean package -DskipTests"
30      }
31    }
32
33    stage("SonarQube Analysis") {
34      steps {
35        withSonarQubeEnv('sonarqube') {
36          sh ""
37          mvn sonar:sonar \
38            -Dsonar.projectKey=sabear-customer-app \
```

Conclusion

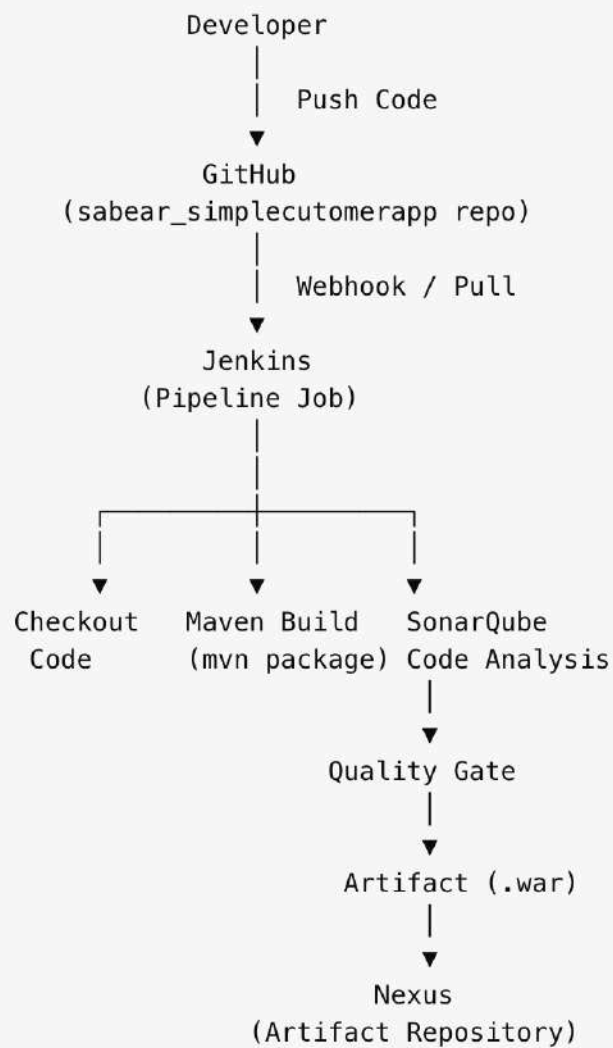
The Jenkins Declarative Pipeline successfully automated the CI process by integrating:

GitHub → Jenkins → Maven → SonarQube → Nexus

The application was built, analyzed for code quality, and the artifact was stored in the Nexus repository.

CI Pipeline Architecture

</> Code



Task 6 : Create One Declarative Jenkins Job

Source Code

<https://github.com/betawins/Trading-UI.git>

Prerequisites

Ensure the following tools are installed on the Jenkins server:

- Jenkins running
- Git installed
- NodeJS installed
- npm installed
- Internet access to GitHub

Verify Node installation:

```
node -v  
npm -v
```

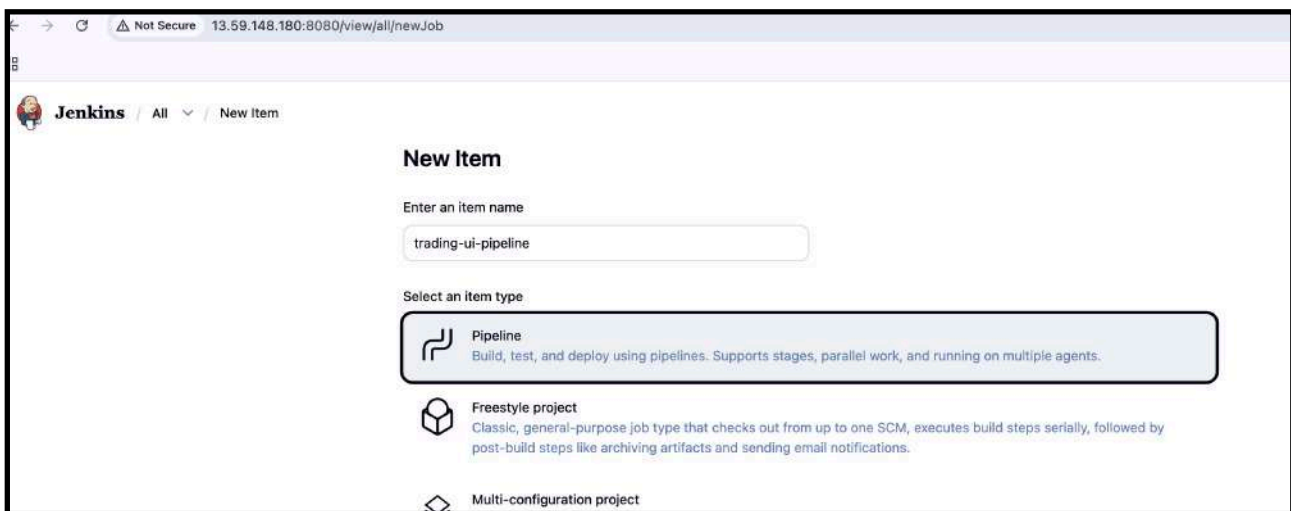
```
Complete:  
[root@ip-172-31-44-77 ~]# node -v  
v18.20.8  
[root@ip-172-31-44-77 ~]# npm -v  
10.8.2  
[root@ip-172-31-44-77 ~]#
```

Step 1 – Create Jenkins Pipeline Job

1. Login to Jenkins.
2. Click **New Item**.
3. Enter the job name:

trading-ui-pipeline

4. Select **Pipeline**.
5. Click **OK**.

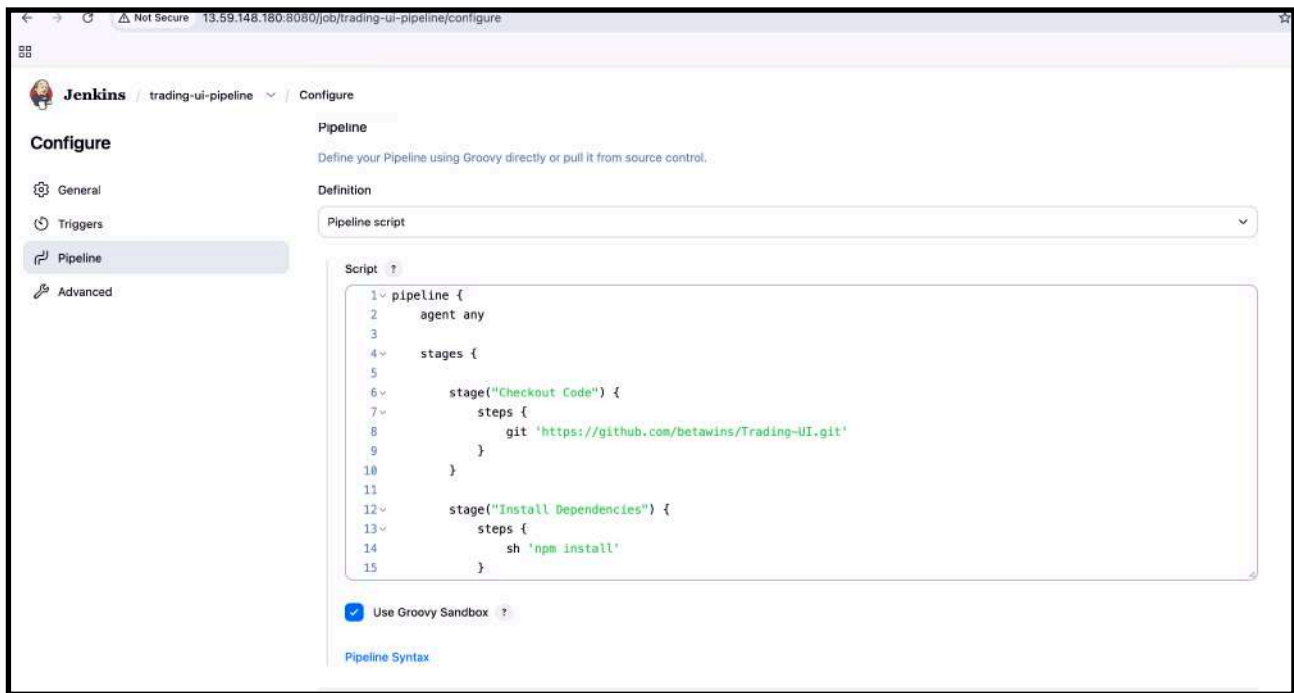


Step 2 – Configure Pipeline Script

Scroll to the **Pipeline** section.

Select:

Pipeline Script



```
pipeline {
  agent any

  stages {

    stage("Checkout Code") {
      steps {
        git 'https://github.com/betawins/Trading-UI.git'
      }
    }

    stage("Install Dependencies") {
      steps {
        sh 'npm install'
      }
    }

    stage("Build Application") {
      steps {
        sh '''
        export NODE_OPTIONS=--openssl-legacy-provider
        export CI=false
        npm run build
        '''
      }
    }
  }

  post {
    success {
      echo "Pipeline Completed Successfully"
    }
    failure {
      echo "Pipeline Failed"
    }
  }
}
```

```
}  
}  
}
```

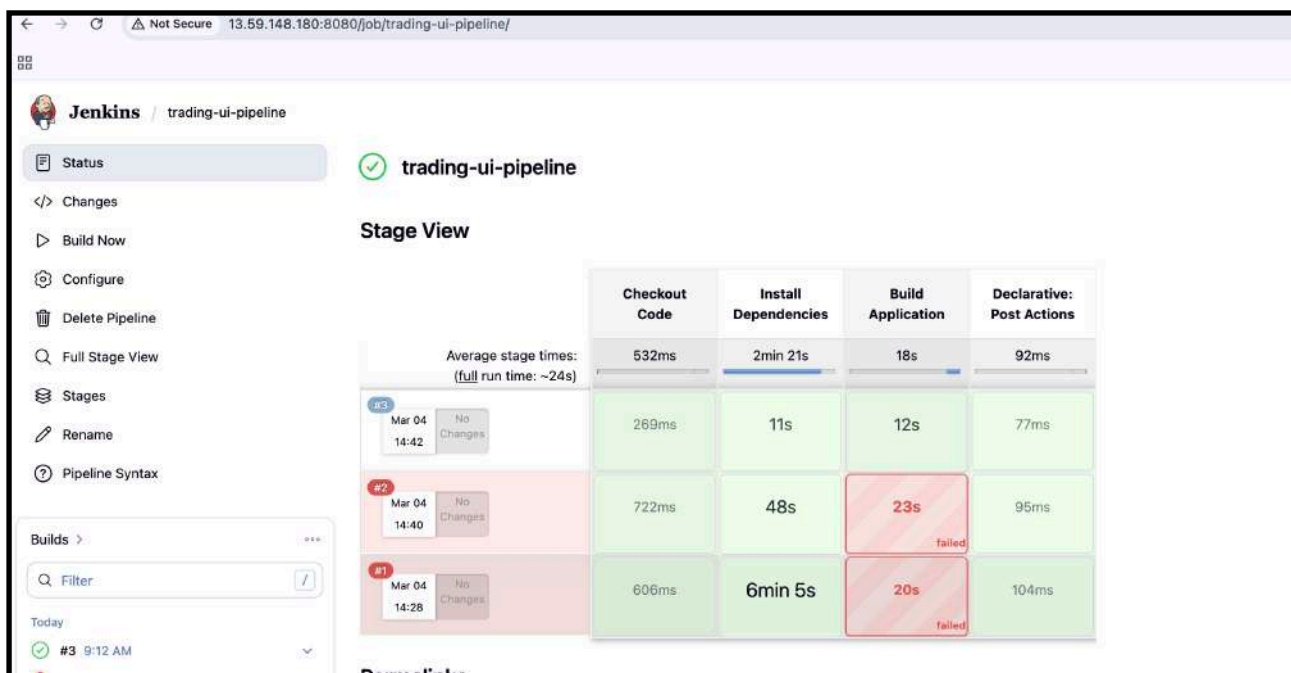
Step 3: Run the Pipeline

Click:

Build Now

Jenkins will execute the pipeline stages:

1. Checkout Code
2. Install Dependencies
3. Build Application



Step : Verify Build Artifact

After successful build, verify the **build folder** is created:

```
cd /var/lib/jenkins/workspace/trading-ui-pipeline
ls
```

You should see:

```
build/
node_modules/
src/
public/
```

The **build/** directory contains the production-ready React application.

```
[root@ip-172-31-44-77 ~]# sudo systemctl start jenkins
[root@ip-172-31-44-77 ~]# cd /var/lib/jenkins/workspace/trading-ui-pipeline
[root@ip-172-31-44-77 trading-ui-pipeline]# ls
'Install nodejs and npn using linux.txt'  Jenkinsfile  README.md  build  coverage  node_modules  package-lock.json  package.json  public  src
[root@ip-172-31-44-77 trading-ui-pipeline]#
```

i-0b60f82715f0f8ac8 (jenkins)

PublicIPs: 13.59.148.180 PrivateIPs: 172.31.44.77

```
[root@ip-172-31-44-77 trading-ui-pipeline]# ls
'Install nodejs and npn using linux.txt'  Jenkinsfile  README.md  build  coverage  node_modules  package-lock.json  package.json  public  src
[root@ip-172-31-44-77 trading-ui-pipeline]# cd src
[root@ip-172-31-44-77 src]# ls
App.css  App.js  App.test.js  Components  Utils  __tests__  config.json  index.css  index.js  logo.svg  serviceWorker.js  setupTests.js
[root@ip-172-31-44-77 src]#
```

Final Verification

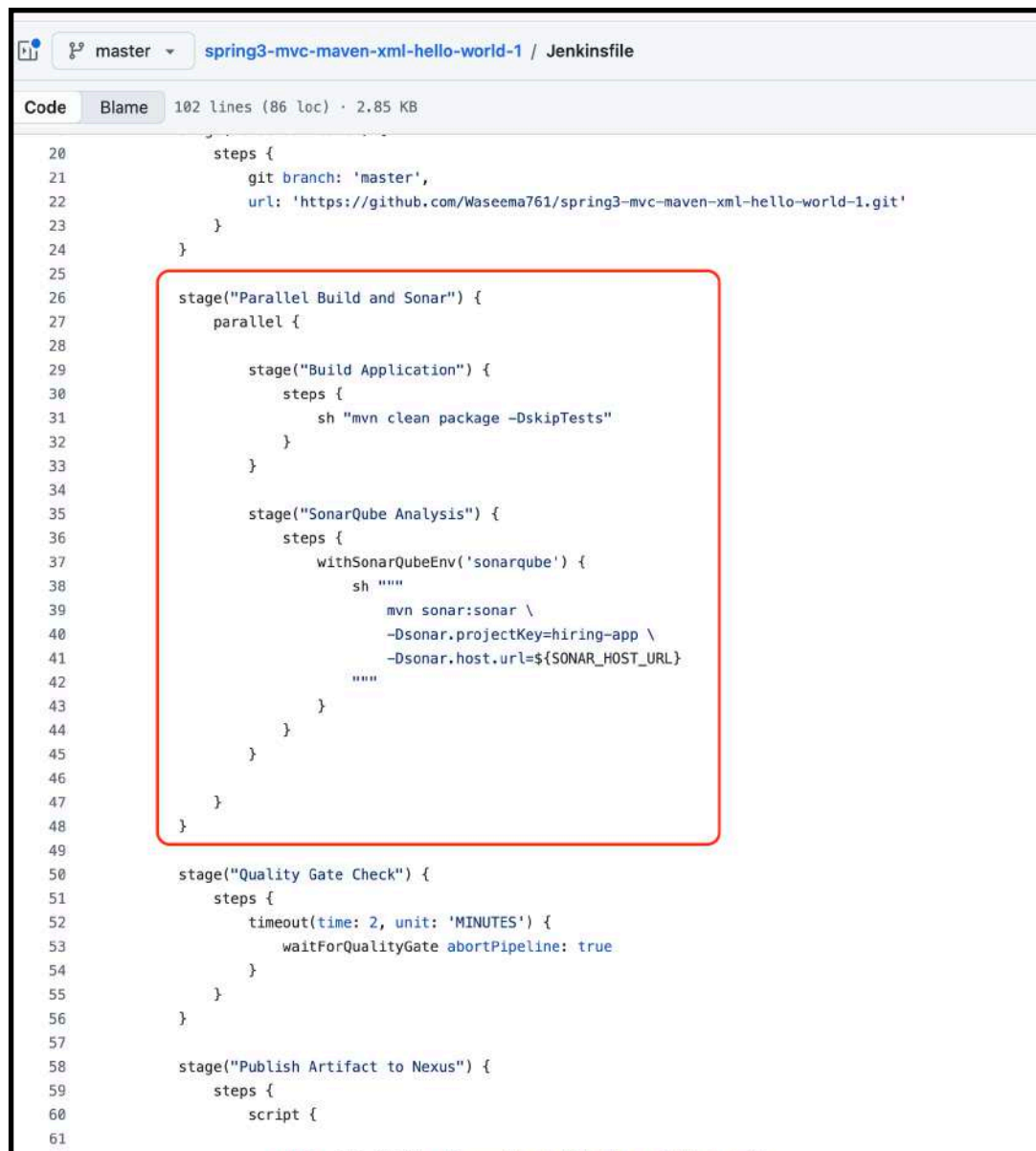
- ✓ Jenkins pipeline executed successfully
- ✓ Source code cloned from GitHub repository
- ✓ npm dependencies installed successfully
- ✓ React application built successfully
- ✓ Build artifacts generated in **build/** directory
- ✓ Jenkins pipeline status shows **SUCCESS**

Task 7:

Execute parallel stages using Jenkins pipeline for any sample job

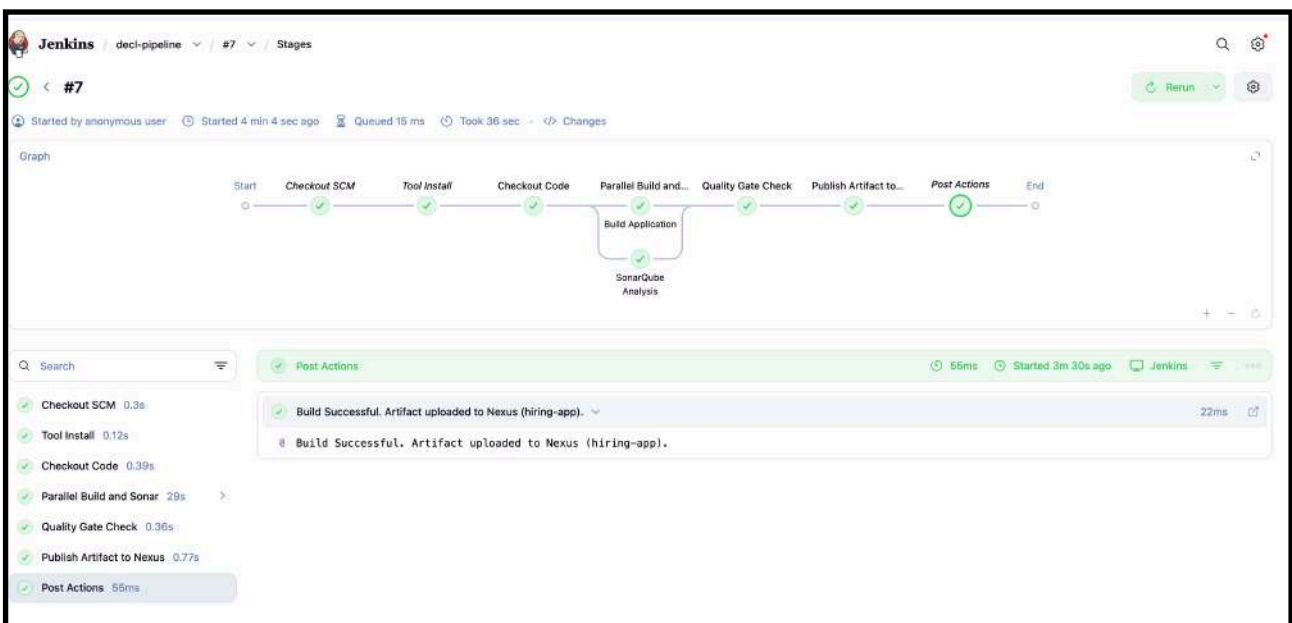
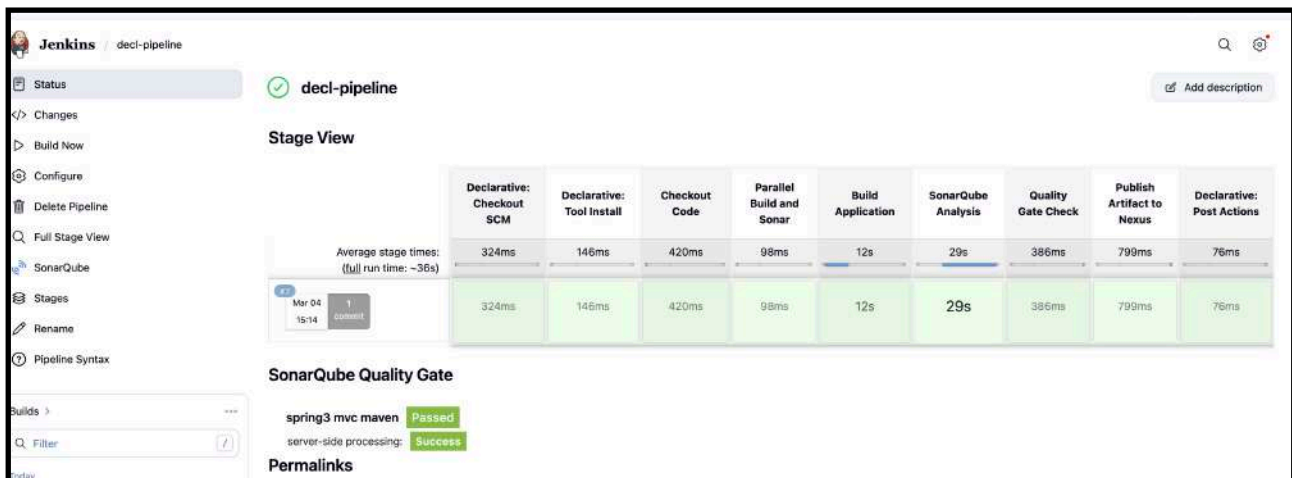
—> lets add the parellel pipeline in task 1 pipeline

Edit the pipeline add parallel stage



```
104 master spring3-mvc-maven-xml-hello-world-1 / Jenkinsfile
Code Blame 102 lines (86 loc) · 2.85 KB
20     steps {
21         git branch: 'master',
22         url: 'https://github.com/Waseema761/spring3-mvc-maven-xml-hello-world-1.git'
23     }
24 }
25
26 stage("Parallel Build and Sonar") {
27     parallel {
28
29         stage("Build Application") {
30             steps {
31                 sh "mvn clean package -DskipTests"
32             }
33         }
34
35         stage("SonarQube Analysis") {
36             steps {
37                 withSonarQubeEnv('sonarqube') {
38                     sh """
39                         mvn sonar:sonar \
40                         -Dsonar.projectKey=hiring-app \
41                         -Dsonar.host.url=${SONAR_HOST_URL}
42                     """
43                 }
44             }
45         }
46     }
47 }
48
49
50 stage("Quality Gate Check") {
51     steps {
52         timeout(time: 2, unit: 'MINUTES') {
53             waitForQualityGate abortPipeline: true
54         }
55     }
56 }
57
58 stage("Publish Artifact to Nexus") {
59     steps {
60         script {
61
```

—> build the job



In the pipeline, the **parallel block** allowed two stages to run at the same time instead of one after another.

These two stages were:

1. **Build Application**
2. **SonarQube Analysis**

Normally Jenkins runs stages **sequentially**, like this:

Checkout Code
↓
Build Application
↓
SonarQube Analysis
↓
Quality Gate
↓
Publish to Nexus

So SonarQube would start **only after the build finished**.

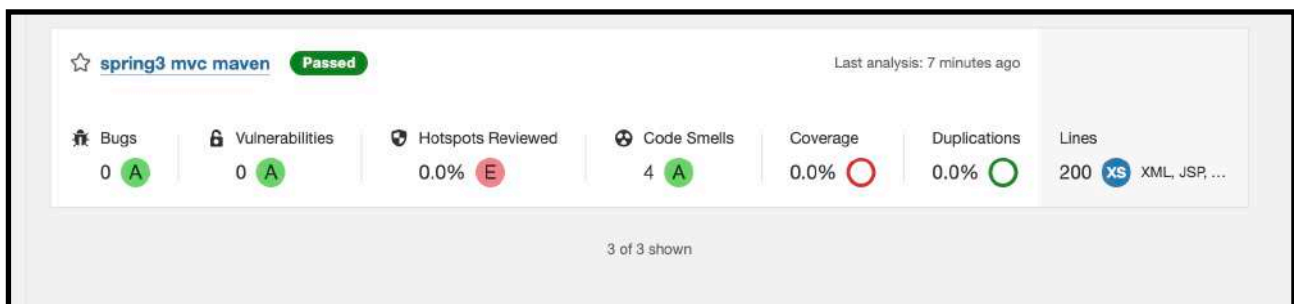
After Adding **parallel**

With the **parallel** block, Jenkins executed **Build Application and SonarQube Analysis simultaneously**.

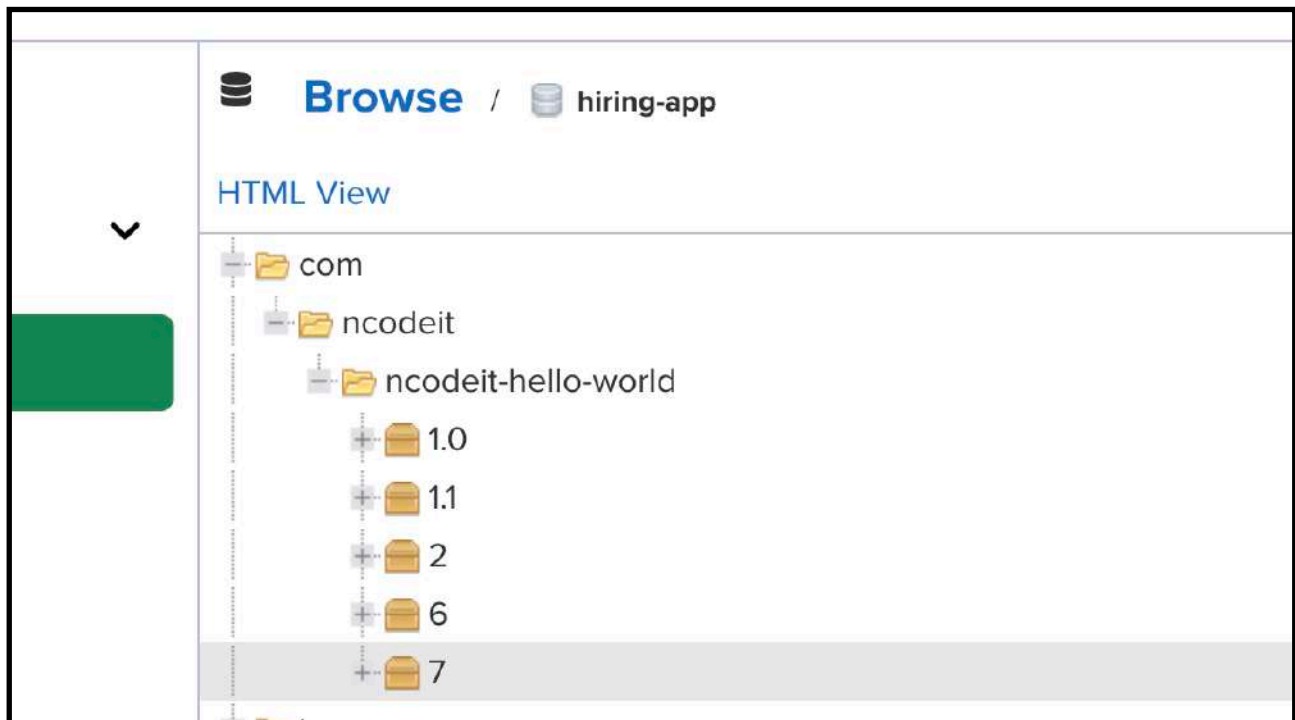
Execution flow became:

Checkout Code
↓
├── Build Application
└── SonarQube Analysis
 (running at the same time)
↓
Quality Gate Check
↓
Publish Artifact to Nexus

Verify:
sonarqube



Nexus:



What Happened During Execution

When the pipeline reached the **Parallel Build and Sonar** stage:

- Jenkins created **two executors/threads**.
- One executor started **Maven build**.
- Another executor started **SonarQube code analysis**.
- Both tasks ran **at the same time**.

After **both stages completed**, Jenkins moved to the next stage:

Quality Gate Check

Benefit of Parallel Execution

The main advantage is **reduced pipeline execution time**.

Instead of:

Build time + Sonar time

the pipeline time becomes roughly:

$\max(\text{Build time}, \text{Sonar time})$

So the pipeline finishes **faster and more efficiently**.

One-Line Explanation (Best for Interview)

The `parallel` block allowed the **Build Application** and **SonarQube Analysis** stages to run **simultaneously**, reducing pipeline execution time and improving CI efficiency.

Task: 8

Execute Jenkins Pipeline Stages Using Shared Libraries

1. Execute Jenkins pipeline stages using shared libraries
2. Reference: <https://phoenixnap.com/kb/jenkins-shared-library>

What Is a Shared Library in Jenkins?

A shared library in Jenkins is a collection of Groovy scripts shared between different Jenkins jobs. To run the scripts, they are pulled into a Jenkinsfile.

Each shared library requires users to define a name and a method of retrieving source code. These methods include local files, **Git repositories**, and Jenkins SCM plugins. Optionally, users can also set a default library version.

Objective

Use a **Jenkins Shared Library** to reuse pipeline functions such as build or deployment steps across multiple Jenkins pipelines. Shared libraries help avoid repeating the same pipeline code and improve maintainability.

How to Create a Shared Library in Jenkins?

Creating a shared library in Jenkins involves the following steps.

Step 1: Create a Groovy Script

First, create a Groovy script using the code you want to save to the library.

For example, the following Groovy script creates a greeting function that prints **Hello** and a user-defined **name** parameter as the output:

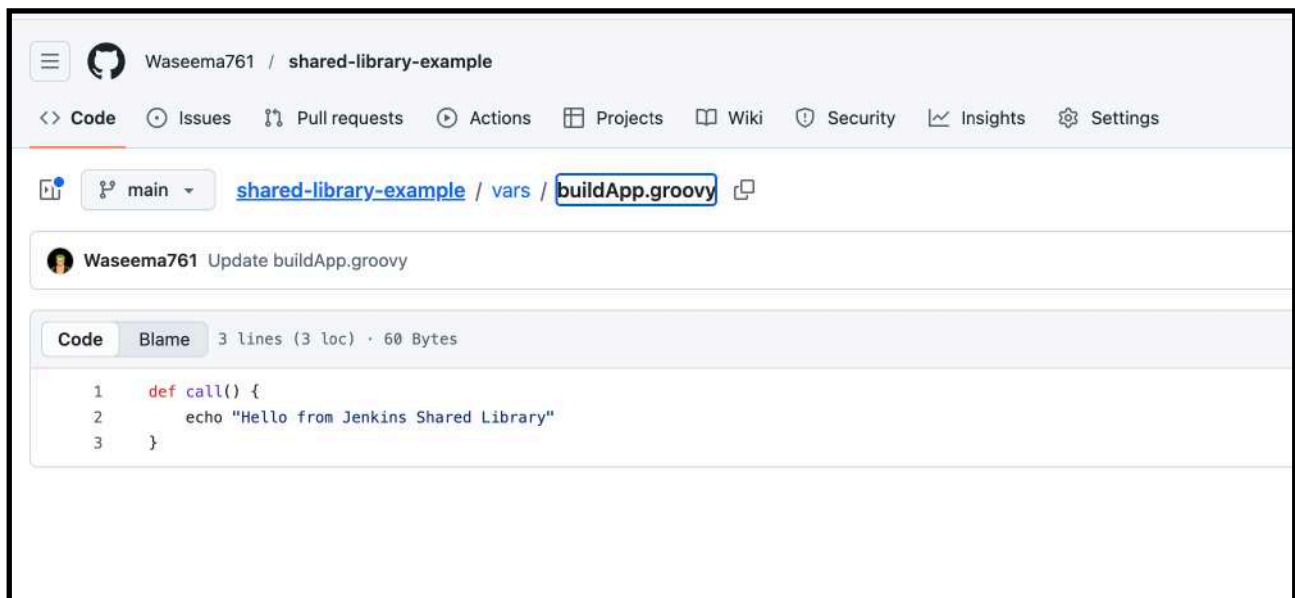
Create Shared Library Repository

Create a GitHub repo like:

```
shared-library-example
```

Structure must be like this:

```
shared-library-example
├── vars
│   └── buildApp.groovy
```



Configure Library in Jenkins

Go to:

Manage Jenkins

→ System

→ Global Trusted Pipeline Libraries

Click **Add**

Fill like this:

Library

Name ?
my-shared-library

Default version ?
main

Currently maps to revision: 2b7fed7e4b302639e9a33c294634f250f4894379

☐ Load implicitly ?

☒ Allow default version to be overridden ?

☒ Include @Library changes in job recent changes ?

☐ Cache fetched versions on controller for quick retrieval ?

Retrieval method
Modern SCM

Loads a library from an SCM plugin using newer interfaces optimized for this purpose. The recommended option when available.

Source Code Management
Git

Project Repository ?
https://github.com/Waseema761/shared-library-example.git

Credentials ?
- none -

Create Jenkins Pipeline Job

Create new job

New Item

→ Pipeline

→ Name: library-job

Jenkins / All / New Item

New Item

Enter an item name
library-job

Select an item type

 **Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 **Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

,Write Very Simple Pipeline

Jenkinsfile:

```
@Library('my-shared-library') _

pipeline {
    agent any

    stages {

        stage('Run Library') {
            steps {
                buildApp()
            }
        }
    }
}
```



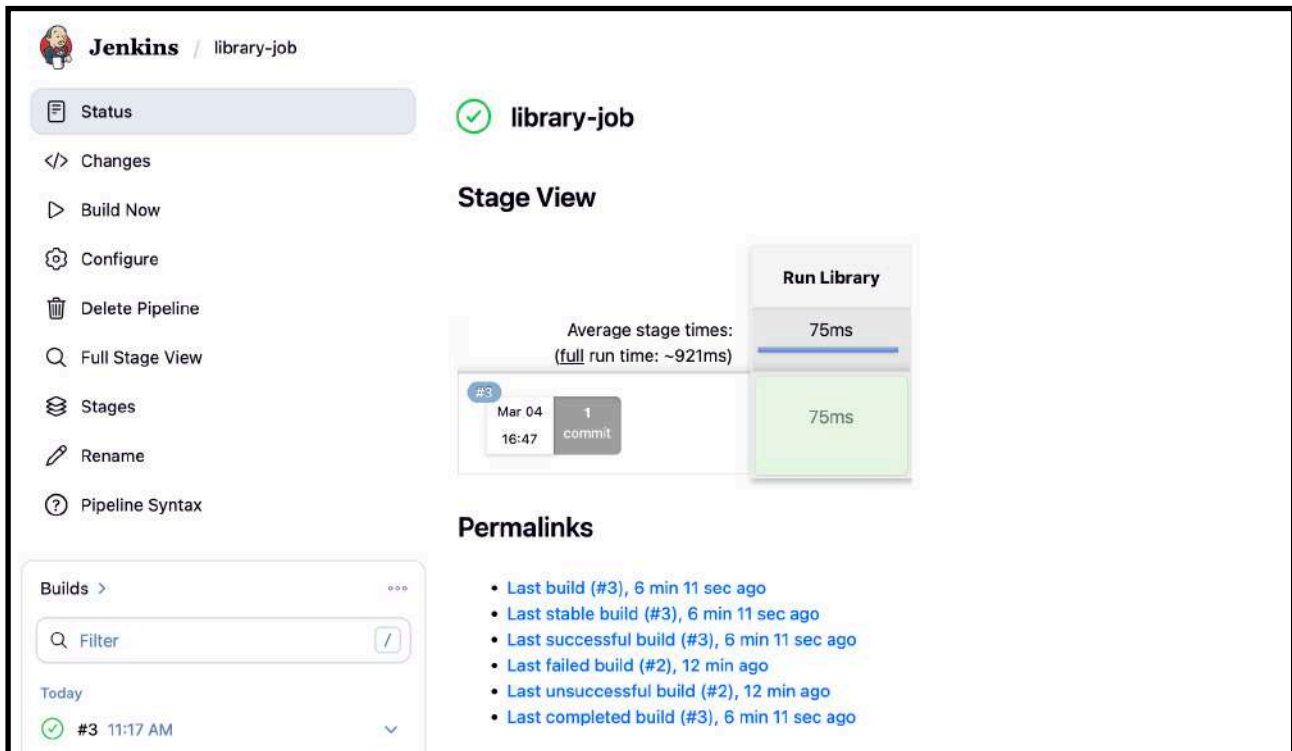
Run the Job

Click:

Build Now

Output will show:

```
Loading library my-shared-library  
Hello from Jenkins Shared Library
```



The screenshot displays the Jenkins web interface for a job named 'library-job'. The left sidebar contains navigation links: Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, Stages, Rename, and Pipeline Syntax. The main content area shows the 'Stage View' for 'library-job', which is in a successful state (indicated by a green checkmark). The 'Run Library' stage is highlighted, showing an average stage time of 75ms and a full run time of approximately 921ms. Below the stage view, a 'Permalinks' section lists various build links, including the last build (#3) and the last stable build (#3). The bottom left corner shows the 'Builds' section with a filter input and a list of builds, including build #3 from 11:17 AM.

Jenkins / library-job

Status

library-job

Stage View

Average stage times:
(full run time: ~921ms)

Run Library

75ms

75ms

Permalinks

- Last build (#3), 6 min 11 sec ago
- Last stable build (#3), 6 min 11 sec ago
- Last successful build (#3), 6 min 11 sec ago
- Last failed build (#2), 12 min ago
- Last unsuccessful build (#2), 12 min ago
- Last completed build (#3), 6 min 11 sec ago

Builds >

Filter

Today

#3 11:17 AM

—> check the console output

```
> git checkout -f 2b7fed7e4b302639e9a33c294634f250f4894379 # timeout=10
Commit message: "Update buildApp.groovy"
> git rev-list --no-walk f49fdb519e5651f5252f26efeb761debff67473e # timeout=10
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/library-job
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Run Library)
[Pipeline] echo
Hello from Jenkins Shared Library
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

What This Task Demonstrates

The Jenkins pipeline:

1. Loaded the **shared library from GitHub**
2. Accessed the **function defined in the library**
3. Executed the function during the pipeline run

This demonstrates how **shared libraries allow reusable pipeline code across Jenkins jobs.**

